EXPERIMENT 13

Simple Programming using CASE and LOOP

Aim:

To learn how to use various MySQL loop statements including case and loop to run a block of code repeatedly based on a condition.

Procedure:

In MySQL, the CASE statement has the functionality of an IF-THEN-ELSE statement and has 2 syntaxes that we will explore.

CASE Syntax

CASE case_value
WHEN when_value THEN statement_list

[WHEN when_value THEN statement_list]

[ELSE statement_list]

END CASE

Procedure:

LOOP Syntax

[begin_label:] LOOP

Statement_list

END LOOP[end_label]

LOOP implements a simple loop construct, enabling repeated execution of the statement list, which consists of one or more statements, each terminated by a semicolon (;) statement delimiter. The statements within the loop are repeated until the loop is terminated. Usually, this is accomplished with a LEAVE statement. Within a stored function, RETURN can also be used, which exits the function entirely.

Questions

1. CASE:
Write a MySQL function using a `CASE` statement to classify a patient's age:

- Age ≤ 18 → **Child**
- Age ≤ 60 → **Adult**
- Otherwise → **Senior Citizen**

2. LOOP:
Write a MySQL function using a `LOOP` and `ITERATE` statement to repeatedly add a patient's age until the total becomes **60 or greater**.

OUTPUT:

1)

```
459    DELIMITER //
460
461 •  CREATE FUNCTION PatientAgeLevel(patient_age INT)
462    RETURNS VARCHAR(20)
463    DETERMINISTIC
464    BEGIN
465        DECLARE age_level VARCHAR(20);
466
467        CASE
468            WHEN patient_age <= 18 THEN
469                SET age_level = 'Child';
470
471            WHEN patient_age <= 60 THEN
472                SET age_level = 'Adult';
473
474            ELSE
475                SET age_level = 'Senior Citizen';
476        END CASE;
477
478        RETURN age_level;
479    END //
480 •  SELECT PatientAgeLevel(25) AS Age_Level;
```

2)

```
488 •  CREATE FUNCTION CalculateAgeTotal(starting_age INT)
489    RETURNS INT
490    DETERMINISTIC
491    BEGIN
492        DECLARE total INT DEFAULT 0;
493
494        age_loop: LOOP
495
496            SET total = total + starting_age;
497
498            IF total < 60 THEN
499                ITERATE age_loop;
500            END IF;
501
502            LEAVE age_loop;
503
504        END LOOP age_loop;
505
506        RETURN total;
507    END //
508
509    DELIMITER ;
510
511 •  SELECT CalculateAgeTotal(20) AS Total_Age;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_Age
60

RESULT:

Thus the Simple programming exercise using CASE and LOOP executed successfully.